



DOCUMENTACIÓN ACADÉMICA Y TÉCNICA

Food Store – Sistema de Gestión de Pedidos

Institución: Universidad Tecnológica Nacional

Carrera: Tecnicatura Universitaria en Programación

Materia: Programación 2

Alumno: Facundo Gaston Vazquez

DNI: 40.717.820

Comisión: 10

Fecha de entrega: 15/06/2026

ÍNDICE

Sección	Página
1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	5
3. Capturas del Sistema	6
4. Dificultades y Soluciones	10
5. Enlaces del Proyecto	11
6. Bibliografía y Webgrafía	11

1. MARCO TEÓRICO

Java como lenguaje de desarrollo

Java es un lenguaje de programación orientado a objetos, de propósito general y fuertemente tipado. Para este proyecto se utilizó Java 21, que es la versión acordada con la cátedra para el desarrollo del trabajo. Una de las características principales que se aprovecharon es la Programación Orientada a Objetos (POO), que permite modelar el dominio del problema en términos de clases, objetos, herencia y polimorfismo.

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en torno a objetos, los cuales combinan datos (atributos) y comportamiento (métodos). Los pilares que se aplicaron en este proyecto son:

- **Encapsulamiento:** cada clase expone solo lo necesario mediante getters y setters, ocultando su implementación interna.
- **Herencia:** todas las entidades del sistema heredan de una clase abstracta llamada Base, que centraliza atributos comunes como id, eliminado y createdAt.
- **Polimorfismo:** se manifiesta en la sobrescritura del método toString() en cada entidad, y en el uso de la interfaz Calculable.
- **Abstracción:** la clase Base es abstracta y no puede instanciarse directamente; obliga a las subclasses a representar entidades concretas del dominio.

Colecciones de Java

Las colecciones son estructuras de datos dinámicas provistas por el lenguaje. En este proyecto se utilizó ArrayList para almacenar en memoria todas las entidades (categorías, productos, usuarios, pedidos y detalles). A diferencia de los arrays simples, los ArrayList permiten crecer o decrecer dinámicamente y ofrecen métodos útiles para recorrer y filtrar elementos.

Manejo de Excepciones

Java provee un mecanismo de manejo de errores basado en excepciones. En este proyecto se crearon excepciones propias que extienden RuntimeException, lo que permite identificar errores del dominio de forma clara y descriptiva, como por ejemplo EntidadNoEncontradaException, StockInvalidoException o EntidadDuplicadaException.

Interfaces

Una interfaz en Java define un contrato que una clase puede implementar. En este proyecto se definió la interfaz `Calculable`, que obliga a la clase `Pedido` a implementar el método `calcularTotal()`. Esto permite estandarizar el comportamiento de cálculo de totales de forma desacoplada.

Enumeraciones (Enums)

Los enums son tipos especiales que representan un conjunto fijo de constantes. Se utilizaron tres: `Rol` (`ADMIN`, `USUARIO`), `Estado` (`PENDIENTE`, `CONFIRMADO`, `TERMINADO`, `CANCELADO`) y `FormaPago` (`TARJETA`, `TRANSFERENCIA`, `EFFECTIVO`). Su uso evita el uso de strings sueltos en el código y mejora la legibilidad.

Soft Delete (Baja Lógica)

En lugar de eliminar físicamente los objetos de la colección, se optó por marcar un atributo `eliminado` = `true`. Esto permite mantener el historial de datos y evitar inconsistencias, por ejemplo cuando un producto ya eliminado sigue siendo parte de un detalle de pedido antiguo.

Almacenamiento en Memoria vs. Base de Datos Relacional

En el contexto de la materia Programación 2, el plan de estudios contempla el uso de JDBC (Java Database Connectivity) como mecanismo para conectar aplicaciones Java con bases de datos relacionales. JDBC es una API estándar de Java que permite ejecutar consultas SQL, insertar, modificar y eliminar registros en una base de datos desde el código Java, sin depender de un motor específico.

Una base de datos relacional organiza la información en tablas relacionadas entre sí mediante claves primarias y foráneas. En un sistema como Food Store, cada entidad corresponde a una tabla, y las relaciones entre ellas se representan con claves foráneas, por ejemplo el campo `usuario_id` en la tabla de pedidos.

Sin embargo, en esta entrega el sistema funciona exclusivamente en memoria utilizando colecciones de Java. Esto significa que los datos no se persisten entre ejecuciones del programa. Esta decisión responde al alcance definido para esta etapa del trabajo, donde el foco está puesto en la correcta aplicación de los principios de POO, el manejo de excepciones y la organización del código por capas. La integración con una base de datos relacional mediante JDBC queda fuera del alcance de esta entrega.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

Organización del proyecto por capas

El proyecto se organizó en cinco capas bien diferenciadas, siguiendo una arquitectura simple pero ordenada:

- **entities/**: contiene las clases del modelo de dominio (Base, Categoria, Producto, Usuario, Pedido, DetallePedido). Ninguna de estas clases contiene lógica de negocio ni acceso al menú.
- **enums/**: contiene los tipos enumerados Rol, Estado y FormaPago.
- **exception/**: contiene todas las excepciones propias del sistema.
- **service/**: contiene la lógica de negocio y validaciones. Cada entidad tiene su propio service (CategoriaService, ProductoService, UsuarioService, PedidoService).
- **ui/**: contiene los menús de consola (MenuPrincipal, CategoriaMenu, ProductoMenu, UsuarioMenu, PedidoMenu) y la clase utilitaria ConsolaUtils.

Esta separación hace que el **Main** solo se encargue de instanciar los servicios y lanzar el menú principal, sin contener ninguna regla de negocio.

Clase Base como superclase común

Se decidió centralizar los atributos **id**, **eliminado** y **createdAt** en una clase abstracta llamada **Base**. Todas las entidades heredan de ella, lo que evita repetición de código y garantiza consistencia en todo el modelo.

Generación de IDs en memoria

Como el sistema no usa base de datos, los IDs se generan mediante un contador (**nextId**) que se incrementa en cada service cada vez que se crea una nueva entidad. Esto simula el comportamiento de un autoincrement de base de datos.

Dependencia entre servicios

Los servicios se comunican entre sí mediante inyección por constructor o por setter. Por ejemplo, ProductoService recibe un CategoriaService en su constructor para poder validar que la categoría existe al crear un producto. De la misma manera, CategoriaService recibe un ProductoService via setter para verificar si una categoría tiene productos activos antes de eliminarla.

Validaciones en el Service, no en el menú

Todas las validaciones de negocio (precio no negativo, mail único, stock suficiente, etc.) se realizan dentro de los services, lanzando excepciones propias. Los menús simplemente capturan esas excepciones con try-catch y muestran el mensaje de error al usuario, sin contener lógica de negocio.

ConsolaUtils como clase utilitaria

Se creó la clase ConsolaUtils con métodos estáticos para leer distintos tipos de datos desde la consola tanto en versión obligatoria como opcional. Esto evita duplicar el manejo del Scanner y los try-catch de parseo en cada menú.

Interfaz Calculable

La interfaz Calculable define el método calcularTotal(). La clase Pedido la implementa recorriendo sus DetallePedido y sumando los subtotales. Esto se llama explícitamente al finalizar la creación de un pedido.

3. CAPTURAS DEL SISTEMA

CAPTURA 1 — Menú principal

“Ejecutamos el proyecto con “run project”

```
run:

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione:
```

CAPTURA 2 — Crear una categoría

```
=== CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Frutas
Descripcion: Frutas
Categoria creada correctamente. ID generado: 1
```

CAPTURA 3 — Listar categorías

```
=== CATEGORIAS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

Listado de categorias:
Categoria{id=1, nombre='Frutas', descripcion='Frutas'}
```

CAPTURA 4 — Crear un producto

```
Categorias disponibles:
Categoria{id=1, nombre='Frutas', descripcion='Frutas'}
Nombre: Naranja
Descripcion: Fruta cítrica fresca
Precio: 1200
Stock: 20
Imagen: naranja.jpg
Disponible? (S/N): S
ID de categoria: 1
Producto creado correctamente. ID generado: 1
```

CAPTURA 5 — Crear un usuario

```
=== USUARIOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Facundo
Apellido: Vazquez
Mail: fvazquez@gmail.com
Celular: 124356273
Contraseña: 1234
Roles:
1. ADMIN
2. USUARIO
Seleccione rol: 1
Usuario creado correctamente. ID generado: 1
```

CAPTURA 6 — Crear un pedido con detalles

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Editar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 2

Usuarios disponibles:
Usuario{id=1, nombre='Facundo', apellido='Vazquez', mail='fvazquez@gmail.com', celular='124356273', rol=ADMIN}
ID de usuario: 1
Formas de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Seleccione forma de pago: 2

Productos disponibles:
Producto{id=1, nombre='Naranja', precio=1200.0, stock=20, disponible=true, categoria=Frutas}
ID de producto: 1
Cantidad: 5
Desea agregar otro producto al pedido? (S/N): n
Pedido creado correctamente. ID generado: 1
Total: 6000.0
```


CAPTURA 7 — Listar pedidos

```
=== PEDIDOS ===
1. Listar
2. Crear
3. Editar estado / forma de pago
4. Eliminar
0. Volver
Seleccione: 1

Listado de pedidos:
Pedido{id=1, fecha=2026-06-09, usuario=Facundo Vazquez, estado=PENDIENTE, formaPago=TRANSFERENCIA, total=6000.0, cantidadDetalles=1}
  DetallePedido{id=1, producto=Naranja, cantidad=5, subtotal=6000.0}
```

CAPTURA 8 — Error de validación

```
=== PRODUCTOS ===
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2

Categorias disponibles:
Categoria{id=1, nombre='Frutas', descripcion='Frutas'}
Nombre: Uva
Descripcion: Uva fresca por racimo
Precio: 560
Stock: -10
Imagen: uva.jpg
Disponible? (S/N): S
ID de categoria: 1
Error: El stock del producto no puede ser negativo.
```

CAPTURA 9 — Baja lógica

```
Listado de categorias:
Categoria{id=1, nombre='Frutas', descripcion='Frutas'}
Categoria{id=2, nombre='Verduras', descripcion='Verduras'}
ID de la categoria a eliminar: 2
Confirma la eliminacion? (S/N): s
Categoria eliminada correctamente.
```

4. DIFICULTADES Y SOLUCIONES

Dependencia circular entre *CategoriaService* y *ProductoService*

Una de las primeras complicaciones que surgió fue la relación bidireccional entre *CategoriaService* y *ProductoService*. *ProductoService* necesita a *CategoriaService* para validar que la categoría existe al crear un producto, pero *CategoriaService* también necesita a *ProductoService* para verificar si una categoría tiene productos activos antes de eliminarla. Si ambos se inyectaban por constructor, se producía una dependencia circular que impedía instanciar los objetos correctamente.

La solución fue inyectar *ProductoService* dentro de *CategoriaService* a través de un setter, en lugar de hacerlo por constructor. Esto permite que primero se construya *CategoriaService*, luego *ProductoService* (que sí recibe *CategoriaService* en su constructor), y finalmente se le asigna el *ProductoService* al *CategoriaService* mediante el setter. Este orden se resuelve en el Main.

Mantenimiento de la consistencia del pedido ante errores

Al crear un pedido con múltiples detalles, si uno de los productos no tiene stock suficiente o genera algún error, el pedido no debe quedar a medias en la colección. El pedido se construye en memoria primero sin agregarlo aún a la colección, y recién al final, cuando todos los detalles se procesaron sin errores, se agrega a la lista. Si ocurre una excepción en algún detalle intermedio, el pedido incompleto simplemente se descarta.

Generación de IDs para los *DetallePedido*

Al principio no era evidente cómo asignar IDs a los detalles de pedido, ya que estos no se crean desde un menú propio sino desde el service de pedidos. La solución fue mantener un contador *nextDetalleId* dentro del mismo *PedidoService*, y asignar los IDs a los detalles una vez que el pedido fue construido exitosamente, antes de agregarlo a la colección.

Validaciones opcionales en la edición

En las operaciones de edición, el usuario puede dejar campos en blanco para conservar el valor anterior. Esto implicó distinguir entre "el usuario no quiso cambiar este campo" (null) y "el usuario ingresó un valor vacío". Para manejarlo de forma consistente, *ConsolaUtils.leerTextoOpcional()* devuelve null cuando el usuario presiona Enter sin escribir nada, y los services solo actualizan el campo si el valor recibido no es null.

Comprensión del soft delete en listados e historial

Al principio puede parecer que el soft delete complica los listados, porque hay que filtrar en cada consulta los objetos marcados como eliminados. Sin embargo, esta decisión es importante para mantener la integridad histórica: si se elimina físicamente un producto de la colección, todos los DetallePedido que lo referencian quedarían apuntando a un objeto inexistente. Con soft delete, el producto sigue en memoria y los pedidos históricos pueden consultarse sin problemas.

5. ENLACES DEL PROYECTO

Repositorio de GitHub:

<https://github.com/facuvazquez1/food-store-java>

Video demostrativo:

https://drive.google.com/file/d/1NFu914BBMljv-DoY24gs3akgagPYwWkX/view?usp=drive_link

6. BIBLIOGRAFÍA Y WEBGRAFÍA

- Oracle. (2024). Documentación oficial de Java SE 21 – ArrayList (java.util). Recuperado de <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html>
- Oracle. (2024). The Java Tutorials – Interfaces y herencia en Java. Recuperado de <https://docs.oracle.com/javase/tutorial/java/landI/index.html>
- Oracle. (2024). The Java Tutorials – Excepciones en Java: creación de excepciones propias. Recuperado de <https://docs.oracle.com/javase/tutorial/essential/exceptions/creating.html>
- Oracle. (2024). The Java Tutorials – Enumeraciones (Enum Types). Recuperado de <https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>
- Oracle. (2024). Java SE 21 – Clase LocalDateTime (java.time). Recuperado de <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/LocalDateTime.html>
- Bloch, J. (2018). Effective Java (3.^a ed.). Addison-Wesley. (Ítem 17: minimizar la mutabilidad; Ítem 41: usar interfaces para definir tipos)
- Deitel, P. y Deitel, H. (2020). Java: Cómo programar (11.^a ed.). Pearson Educación. (Capítulos 9, 11 y 16: POO, excepciones y colecciones)
- Apache NetBeans. (2024). Guía de inicio rápido con NetBeans IDE para Java. Recuperado de <https://netbeans.apache.org/tutorial/main/kb/docs/java/quickstart/>